
Saarthi

Phone-First Personal-Loan Application Assistant

Implementation-aligned MVP design

Updated 8 September 2026

A doubt-aware, state-consistent voice experience that lets a borrower ask questions, correct information, interrupt speech, and resume a pre-application without corrupting the draft.

Product boundary	Fictional personal-loan pre-application using synthetic, non-sensitive data
Primary proof	Voice remains useful and controllable when the borrower leaves the expected answer path
Technical character	Guarded AI pipeline with deterministic state, calculations, safety gates, and evidence
Terminal outcome	Reviewable draft plus user-marked demo completion; never lender submission, approval, KYC, mandate, or disbursal

Contents

1	Executive Summary	1
1.1	The user problem	1
1.2	The proposed experience	1
1.3	Why voice is essential	1
1.4	What the prototype must prove	1
2	Product Scope and Technical Requirements	2
2.1	Target user and scenario	2
2.2	Application scope	2
2.3	Six technical requirements	2
2.4	Non-negotiable product boundaries	3
3	High-Level Architecture	3
3.1	Architecture at a glance	3
3.2	Verified production topology	4
3.3	Primary runtime path	6
3.4	Component responsibilities	6
3.5	Storage and provider boundaries	7
4	Information and State Models	7
4.1	Four sources of truth	7
4.2	Application Information Model	7
4.2.1	Logical aggregate	7
4.2.2	ApplicationDraft root	9
4.2.3	CommittedFieldValue	9
4.2.4	FinancialProjection	9
4.2.5	Change history and output artifact	9
4.2.6	Atomic mutation contract	10
4.3	Conversation State Model	10
4.4	Three independent versions	10
4.5	Resume and heard-output contracts	11
5	Decision Logic and Runtime Behaviour	13
5.1	Typed turn events	13
5.2	Cancel-first protocol	13
5.3	Grounded explanation decision	13
5.4	Financial speech decision	13
5.5	Neutrality and action decision	14

6	Safety and Fallback Framework	14
6.1	Safety invariants	14
6.2	Progressive fallback choices	14
6.3	Failure decisions	15
7	Research Evidence and Design Choices	17
7.1	How evidence is used	17
7.2	Literature basis for evaluated alternatives	17
7.3	Experiment summary	17
7.4	Implementation reconciliation after live integration	18
7.5	Decisions carried into the MVP	18
7.6	Important rejected alternatives	19
8	Evaluation and Acceptance	21
8.1	Evaluation principle	21
8.2	What is tested and how	21
8.3	Performance and reliability targets	22
8.4	Observed implementation snapshot	22
8.5	Canonical scenario set	23
8.6	End-to-end demonstration	24
8.7	Verdict	24
9	Observability and Demonstration Evidence	26
9.1	Why observability is part of the product proof	26
9.2	Minimum event envelope	26
9.3	Dashboard panels	26
10	Limitations and Future Work	29
10.1	Claims deliberately not made	29
10.2	Future work	29
A	Implementation Low-Level Design	30
A.1	Purpose and fixed boundaries	30
A.2	Runtime processes and deployment units	30
A.3	Component contracts	32
A.4	Core message contracts	32
A.5	Session creation and API surface	33
A.6	One finalized-turn execution	34
A.7	Persistence, cache, and retrieval design	36
A.8	Concurrency, retry, and idempotency rules	37
A.9	Configuration and secret ownership	37
A.10	Implemented package layout and implementation order	38

1 Executive Summary

1.1 The user problem

Personal-loan applications combine unfamiliar financial vocabulary with a long sequence of questions. A first-time or financially less-confident borrower may need to ask what a field means, clarify a fee, correct an earlier answer, or stop an explanation midway. Conventional sequential forms make the borrower leave the application to seek help, while rigid voice systems may mistake a doubt for an answer, lose the pending question, alter the wrong field, or continue speaking after the user has changed direction.

The product problem is therefore not simply “complete a form by voice.” It is:

Help a borrower complete a bounded loan pre-application through voice while preserving control, context, and data correctness whenever the conversation leaves the expected path.

1.2 The proposed experience

The assistant guides one fictional application field at a time. During the same conversation, the borrower can:

- answer fields in natural language;
- ask contextual or product-specific questions;
- correct the current field or an earlier answer;
- pause, resume, repeat, go back, or stop;
- hear a grounded explanation and a listener-oriented financial summary; and
- review a written draft and fact sheet, raise a demonstration help ticket, and manually mark the demo journey complete.

The system remains neutral and draft-only. It does not recommend a loan, optimize for conversion, collect real sensitive information, or expose an action that could submit to a lender or approve an application. The final browser button is a user-controlled demonstration status: it stops the voice worker and shows that the draft was completed, but it sends no lender request and has no financial effect.

1.3 Why voice is essential

Voice is not an ornamental input mode in this design. The main proof depends on spoken collection, audible explanation, interruption during playback, and accurate resumption based on what the borrower actually heard. Removing speech would remove the central interaction problem: handling barge-in, partial delivery, listener-oriented financial speech, and continuous hands-free clarification. The written draft is a review and safety artifact, not a substitute for the voice journey.

1.4 What the prototype must prove

The prototype succeeds when it shows that:

1. a question or command is never stored as an application answer;
2. a doubt is answered without losing the original pending field;
3. a correction changes only its intended field;

4. confirmed values survive interruption and dependency failure;
5. product explanations are supported by approved evidence or deterministic calculations;
6. obsolete speech and delayed results cannot re-enter after a newer turn or control;
7. spoken and written financial values agree exactly; and
8. the final artifact remains visibly a non-lender-submitted draft, with enough trace evidence to reconstruct the demonstration.

2 Product Scope and Technical Requirements

2.1 Target user and scenario

The target is a first-time or financially less-confident borrower who is comfortable speaking but may not understand every loan term. The demonstration uses one synthetic borrower, one fictional product, and a bounded inbound “continue by voice” journey. This positioning avoids claiming that the prototype is suitable for real lending decisions or regulated deployment.

2.2 Application scope

The demonstration collects eight non-sensitive fields, grouped by meaning:

Group	Fields
Loan request	Requested amount, loan purpose, and preferred tenure
Financial profile	Employment type, monthly take-home income, and existing monthly repayments
Application context	City and contact preference; conversational city phrases are normalized to the city value, and no real phone number or email address is collected

These fields are sufficient to prove understanding, correction, calculation, explanation, and review. They are not intended to reproduce a complete lender application.

2.3 Six technical requirements

TR	Capability	Required behaviour
TR-1	Off-path turn understanding	Distinguish answers, doubts, corrections, controls, mixed turns, and ambiguity. Produce a typed proposal; do not grant the model write authority.
TR-2	State-consistent interruption, correction, and resumption	Preserve one pending field, suspend temporary branches, apply corrections atomically, and return to the correct checkpoint.
TR-3	Grounded contextual explanation	Answer product questions only from approved, versioned facts or deterministic calculations; abstain when evidence is incomplete.
TR-4	Faithful controlled financial speech	Render exact value–label pairs as short, listener-oriented segments and use the same structured projection for speech and writing.
TR-5	Persistent user control and cancellation	Give stop, pause, repeat, go-back, and newer turns priority over reasoning, synthesis, queued audio, and playback.

TR	Capability	Required behaviour
TR-6	Neutral and draft-only completion	Block pressure, urgency, hidden costs, unauthorized actions, and any implication of submission, approval, or recommendation.

2.4 Non-negotiable product boundaries

- Only synthetic, non-sensitive data is used.
- There is one fictional product and one approved fact set.
- There is no real CRM, lender, credit-check, KYC, authentication, document, payment, mandate, approval, or disbursal integration.
- The only terminal outcomes are a reviewable completed draft, a user-marked demo completion state, or a cancelled draft; none is a lender submission.
- English and Hindi Rime Coda/Nadi output profiles are configurable at session creation. The acceptance evidence covers the bounded English flow; broader multilingual recognition, code-switching, multiple loan products, production telephony, long-term memory, real human-agent transfer, and production compliance validation remain future work.

3 High-Level Architecture

3.1 Architecture at a glance

The prototype uses a **modular monolith with two runtime processes**: one FastAPI service and one LiveKit agent worker share the same guarded domain modules and authoritative SQL repository. In production, both Railway services reference the same Railway PostgreSQL database; local development retains SQLite through the same asynchronous SQLAlchemy repository contract. The borrower web client is deployed on Vercel and joins a LiveKit Cloud room through a short-lived token issued by FastAPI. Provider adapters isolate Deepgram streaming recognition, Gemini structured interpretation, optional Groq grounded wording, Qdrant Cloud retrieval, and Rime synthesis. Qdrant stores approved product facts; the in-process cache and optional Redis layer reduce repeated retrieval work; PostgreSQL stores workflow checkpoints, application drafts, and append-only trace events used by the dashboard and evaluation.

3.2 Verified production topology

Deployment unit	Platform	Deployed responsibility
Borrower web client	Vercel	React/Vite interface, microphone and playback controls, speaker-labelled transcript, draft review, and evidence dashboard. It calls the API through <code>VITE_API_BASE_URL</code> .
FastAPI service	Railway	Production HTTPS API for sessions, LiveKit tokens and agent dispatch, draft/event/evidence reads, controls, and export. CORS is restricted to the deployed frontend origin.
LiveKit agent worker	Railway	Long-running <code>saarthi-agent start</code> process for room jobs, Deepgram STT, guarded turn processing, Rime TTS, interruption recovery, and delivery events.
Authoritative state and evidence	Railway PostgreSQL	Shared <code>sessions</code> , <code>application_drafts</code> , <code>idempotency_results</code> , and <code>trace_events</code> tables accessed by both runtime processes through one <code>DATABASE_URL</code> .
Realtime room transport	LiveKit Cloud	Room creation, participant transport, worker dispatch, microphone publication, audio subscription, and short-lived scoped tokens.
Approved product knowledge	Qdrant Cloud	Version-filtered embeddings for the <code>loan_product_facts</code> collection, with the bundled approved local fact set as a safe fallback.

The production smoke check on 8 September 2026 returned `status=ok`, `environment=production`, and an empty `missing_provider_configuration` list. An initial worker failure also validated the shared-store boundary: the worker could receive a LiveKit job but could not resolve the API-created session until both Railway services referenced the same PostgreSQL database. This is an operational verification of connectivity and configuration, not a new quality or latency benchmark.

Probabilistic components may interpret, retrieve, or phrase. Deterministic components alone own application writes, state transitions, calculations, cancellation, release permission, and the action boundary.

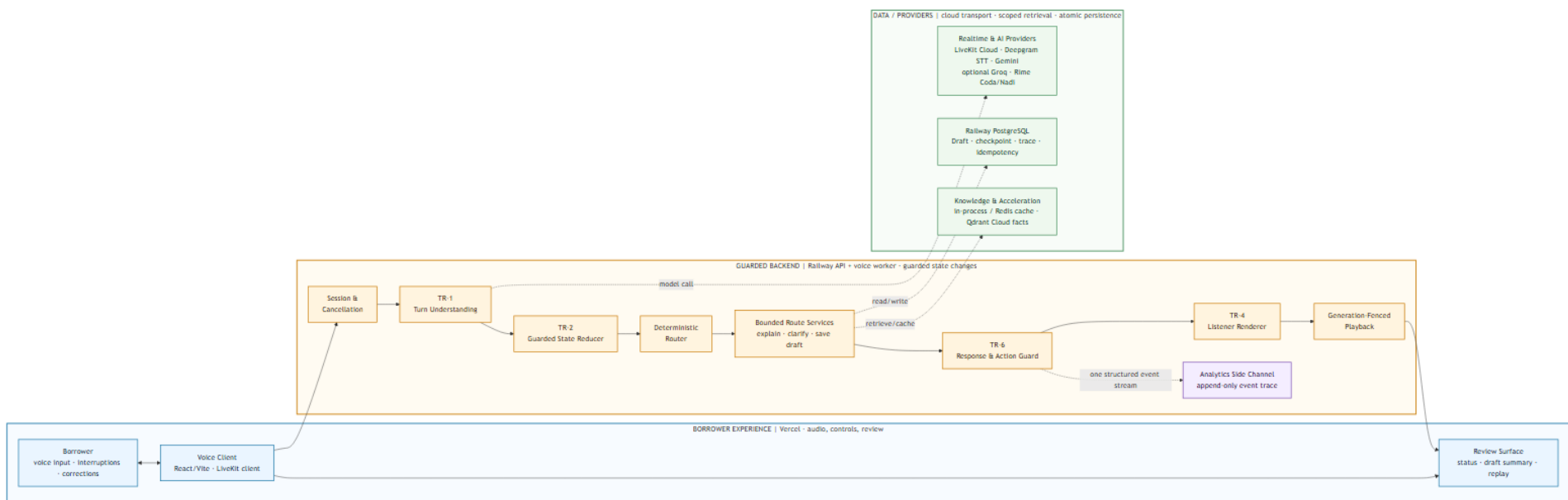


Figure 1: High-level architecture and deployed boundaries. The Vercel borrower client enters the guarded Railway runtime through FastAPI and LiveKit; both runtime processes share Railway PostgreSQL, while provider and knowledge dependencies remain replaceable ports. Analytics receives one structured side-channel event stream.

3.3 Primary runtime path

1. The borrower speaks or issues a priority control through the voice client.
2. Streaming speech recognition returns a finalized transcript tagged with the active session and turn.
3. TR-1 first resolves deterministic controls, hedged values, calculation cues, and unambiguous field answers. Remaining turns go to Gemini with five context-selected examples and return a typed semantic proposal.
4. The guarded reducer checks explicitness, target, normalization, versions, and permissions. It commits one valid application patch or makes no application mutation.
5. A deterministic router selects field handling, grounded explanation, financial calculation, control handling, clarification, or fallback.
6. Product questions retrieve versioned facts through the in-process/optional Redis cache, Qdrant, and bundled approved-fact fallback; arithmetic is routed to deterministic calculation.
7. TR-6 verifies grounding, numerical completeness, neutrality, draft status, and the action allowlist.
8. TR-4 converts approved content into short labelled segments and sends them to Rime.
9. The client keeps one subscribed audio element attached, plays only current-generation audio, and records browser stop measurements; the worker records delivered or interrupted speech segments.
10. Each material decision emits a structured trace event used by the review and dashboard experience.

3.4 Component responsibilities

Component	Responsibility	May change application data?
Voice client	Microphone, playback, transcript, controls, and immediate local audio stop	No
Session and cancellation coordinator	Turn identity, versions, active jobs, deadlines, checkpoints, and output fencing	No
TR-1 interpreter	Deterministic fast paths plus Gemini typed act, route, target, candidate, and uncertainty	No
Guarded reducer	Validates and atomically applies permitted state transitions and application patches	Yes; sole authority
Validators and normalizers	Convert supported expressions into typed values, including conversational city extraction, and return validation evidence	No
Grounded explanation service	Retrieve scoped facts, create claim-evidence links, and abstain when coverage fails	No
Financial calculator	Produce a versioned projection from confirmed inputs and approved product facts	No direct write
Response and action guard	Block unsupported, incomplete, persuasive, or prohibited response plans	No
Listener renderer and Rime adapter	Produce faithful speech segments and stream Rime Coda/Nadi audio at 24 kHz with version tags	No
Stores and event trace	Persist committed draft, checkpoint, approved facts, cache entries, and append-only evidence	Only through owned interfaces

3.5 Storage and provider boundaries

Boundary	Design choice
Application store	Authoritative committed draft and revision; compare-and-set writes only
Conversation checkpoint	Current workflow position, versions, active work, and safe resumption data
Qdrant	Approved product facts with product, status, type, and version metadata filters
Cache layer	In-process query cache with optional Redis backing for reusable approved-fact retrieval; never authoritative
Event trace	Append-only record of interpretation, evidence, transitions, mutations, cancellation, delivery, fallback, and verdict
External providers	Replaceable Deepgram, Gemini, optional Groq, Qdrant, and Rime adapters; no provider receives mutation authority

4 Information and State Models

4.1 Four sources of truth

Source	What it authoritatively answers
Application Information Model	What information has been safely committed to the draft?
Conversation State Model	What is the system doing now, what work is valid, and where should it resume?
Versioned product-fact collection	Which approved facts may support a product claim?
Append-only event trace	What actually happened during the interaction and evaluation?

The in-process/Redis cache is deliberately absent from this list because cached data is replaceable and never authoritative.

4.2 Application Information Model

The Application Information Model is the durable domain record. It contains only safely committed borrower inputs, deterministic derived values, and their provenance. Temporary interpretations, confidence, playback, retrieved passages, and active tool work belong elsewhere.

4.2.1 Logical aggregate

```

ApplicationDraft
|- LoanRequest
|- FinancialProfile
|- ApplicationContext
|- CommittedFieldValue records
|- FinancialProjection
|- FieldChangeRecord history
'- DraftArtifact

```

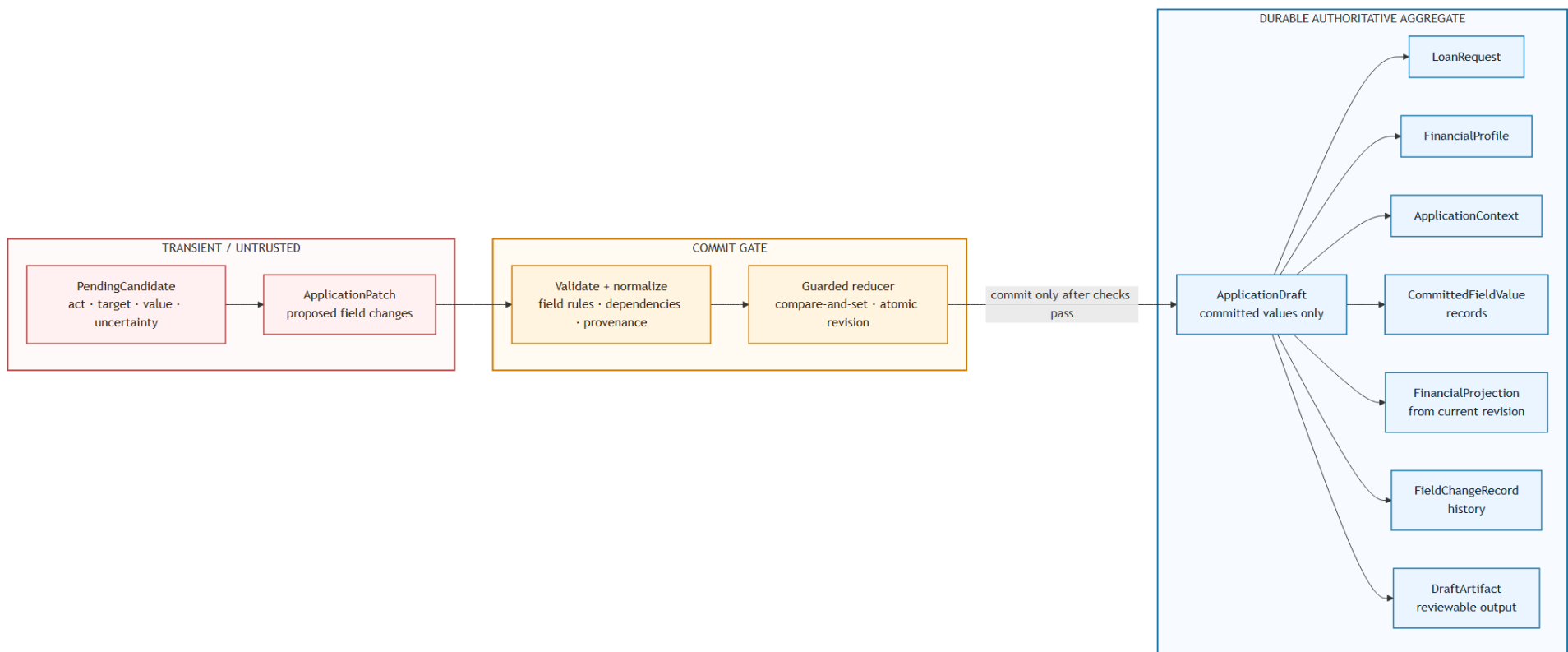


Figure 2: Application aggregate and commit boundary. A PendingCandidate becomes an ApplicationPatch, passes validation and normalization, and reaches the authoritative ApplicationDraft only through the guarded reducer.

4.2.2 ApplicationDraft root

Attribute group	Attributes	Meaning
Identity	<code>draft_id</code> , <code>owner_session_id</code>	Identifies the synthetic draft and its active journey; never a lender application number.
Product and schema	<code>product_id</code> , <code>product_version</code> , <code>schema_version</code>	Pins the approved fact set, validation rules, and calculation context.
Lifecycle	<code>status</code>	One of <code>in_progress</code> , <code>ready_for_review</code> , <code>completed_draft</code> , or <code>cancelled</code> .
Concurrency	<code>revision</code>	Increments exactly once per committed atomic patch.
Content	Loan request, financial profile, application context	The eight committed application values, grouped by meaning.
Derived reference	<code>current_projection_id</code>	References only a projection derived from the current application revision.
Audit	<code>created_at</code> , <code>updated_at</code>	Supports ordering and traceability; never implies submission.

4.2.3 CommittedFieldValue

Each committed field stores a typed authoritative value and durable provenance:

Attribute	Purpose
<code>field_id</code> , <code>typed_value</code>	Stable field identity and normalized committed value
<code>source_turn_id</code> , <code>source_span</code>	The user turn and exact transcript evidence that authorized the value
<code>normalizer_version</code> , <code>validator_version</code>	Reproducible parsing and validation lineage
<code>committed_at_revision</code>	Draft revision at which the value became authoritative
<code>last_change_kind</code> , <code>updated_at</code>	Distinguishes initial entry from correction and records its time

Candidate value, confidence, ambiguity, and confirmation status are not stored here; they remain in the Conversation State Model until accepted or discarded.

4.2.4 FinancialProjection

The projection is deterministic and replaceable. It records its source application revision, product and fact-set version, calculator version, typed inputs, and labelled outputs: requested amount, processing fee, tax, total deduction, net disbursal, EMI, tenure, and total repayment. A dependent field correction invalidates the older projection. One formatter derives both the written fact sheet and the listener-oriented speech plan from the same snapshot.

4.2.5 Change history and output artifact

A `FieldChangeRecord` stores the target field, old and new values, source turn, change type, before/after revisions, and timestamp. Records are append-only. A `DraftArtifact` identifies the exact draft revision and projection represented in the reviewable written output and labels it as non-submitted.

4.2.6 Atomic mutation contract

An `ApplicationPatch` is a command, not application data. The reducer accepts it only when the session and draft match, the expected revision is current, the turn authorizes a write, the field target is explicit, deterministic validation succeeds, required confirmation is present, and the idempotency key has not been used. Acceptance changes one intended field and increments the revision once; rejection produces no partial mutation.

4.3 Conversation State Model

The Conversation State Model is short-term operational memory. It controls the active phase, pending field, current interpretation, resumption, asynchronous work, failure recovery, and what speech the borrower actually heard.

State group	Core attributes	Responsibility
Identity	Session, trace, channel, language	Isolates and correlates one journey.
Lifecycle	Session status and active phase	Separates session lifecycle from collection, explanation, correction, review, and fallback phases.
Workflow position	Pending field, resume checkpoint, navigation stack	Preserves the unfinished task across temporary branches.
Current turn	Turn ID, transcript, acts, route, target, candidate, interpretation status	Holds TR-1's typed proposal without write authority.
Versioning	State version, generation ID, linked application revision	Fences workflow transitions, background work, and stale output.
Active work	Job ID, type, captured versions, status, deadline	Tracks STT, LLM, retrieval, calculator, safety, and TTS jobs.
Grounding	Product/fact version, retrieved fact IDs, calculation ID	Makes current explanations auditable.
Output delivery	Response and segment IDs, queue, playback cursor, delivery status	Distinguishes generated, queued, playing, interrupted, and delivered speech.
Control memory	Last fully heard response/question and last safe checkpoint	Supports repeat, go back, pause, and exact resumption.
Failure	Component, reason, retry count, fallback route	Makes failure bounded and visible without altering confirmed data.

4.4 Three independent versions

Version	Changes when	Protection provided
<code>application_revision</code>	A field is committed or corrected	Rejects conflicting/duplicate data patches and identifies projection inputs.
<code>state_version</code>	An accepted workflow transition occurs	Orders doubt, clarification, navigation, pause, resume, and fallback changes.
<code>generation_id</code>	A newer turn or priority control invalidates outstanding work	Prevents obsolete model, retrieval, TTS, queue, or playback output from re-entering.

Separating these counters means a repeat command can invalidate speech without pretending that application data changed.

4.5 Resume and heard-output contracts

Before a doubt or earlier-field correction begins, the current pending field becomes the resume checkpoint. Temporary work may read the draft but cannot replace that checkpoint. On completion, clarification, timeout, or fallback, the reducer verifies and restores the valid checkpoint. A deliberate go-back command changes navigation; repeat does not.

Every response is divided into ordered segments. Only fully delivered segments enter heard history. An interrupted explanation is not silently treated as shared context, and repeat selects the last safe prompt or explicitly requested segment rather than an unheard remainder.

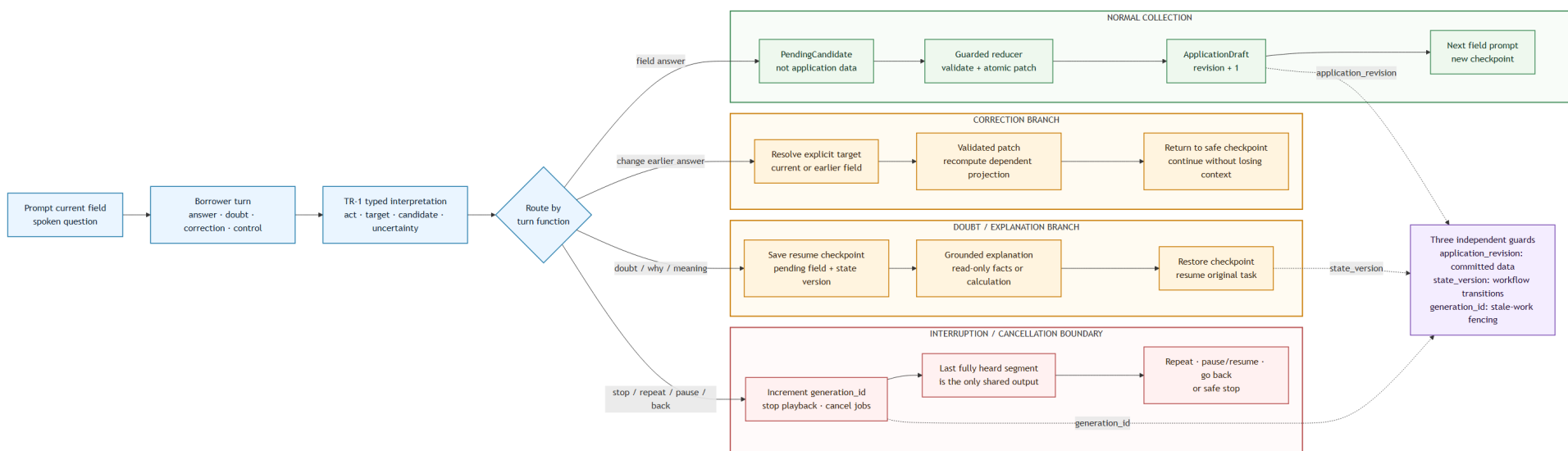


Figure 3: Conversation state, checkpoint, and cancellation boundary. A typed turn is routed into normal collection, a temporary doubt or correction branch, or a control path that fences stale work. Only the guarded reducer changes application data; the three independent versions protect data, workflow, and output delivery.

5 Decision Logic and Runtime Behaviour

5.1 Typed turn events

Event	Application effect	Conversation effect
Resolved answer	Commit one valid field	Advance to the next unanswered field
Answer with doubt	No immediate commit; candidate stays provisional	Explain or clarify, then confirm or discard and resume
Contextual/product doubt	No application mutation	Save checkpoint, answer from approved evidence, return to original field
Earlier-field correction	Patch only the explicit target after confirmation	Return to the previously pending field
Current-field correction	Patch the current target after confirmation	Advance only when the corrected value is valid
Ambiguous/low-confidence turn	No mutation	Ask one focused clarification
Repeat/go back	No value mutation	Replay safe output or change navigation
Pause/resume	No value mutation	Cancel active work, preserve or restore checkpoint, start a fresh generation
Stop/cancel	Preserve committed values; cancel may mark the draft cancelled after confirmation	Stop work and playback, then terminate safely
Dependency failure	No mutation	Apply bounded fallback and return to the valid checkpoint
Asynchronous result	Only its specifically authorized effect, if versions match	Continue the current route
Stale result	No effect	Log and discard without a transition
Journey complete	Mark only a reviewable draft complete	End collection and present review

5.2 Cancel-first protocol

Stop, pause, repeat, go back, correction, and a newer user turn all establish a new generation boundary before replacement work begins. The client stops local playback and clears queued audio; active older jobs are marked obsolete; and every returning result must still match the session, state version, generation, and route. This protects the experience even when an upstream provider cannot cancel an already-running request.

5.3 Grounded explanation decision

Product questions use mandatory product, status, and version filters before ranking. Hinglish aliases and bounded expansion improve retrieval, while deterministic routing separates calculations from descriptive facts. A response plan links every claim to a fact ID or calculation ID. Missing evidence produces clarification or abstention, never a model-memory product claim.

5.4 Financial speech decision

The calculator produces typed, labelled values. The listener renderer expands ambiguous symbols, preserves units, separates dense material into short segments, and verifies fact preservation before Rime

synthesis. Personalized values are not served from reusable static-audio cache entries. Audio chunks carry response, segment, and generation IDs so obsolete playback can be rejected.

5.5 Neutrality and action decision

Prompt instructions are only the first layer. A typed response plan passes deterministic checks for required facts, cost completeness, numerical consistency, pressure language, decline parity, explicit draft status, and allowed actions. Fixed safe templates replace failed plans. The runtime action registry contains only draft updates, approved fact-sheet display, session controls, and local export; real lending actions do not exist.

6 Safety and Fallback Framework

6.1 Safety invariants

ID	Invariant
INV-01	Only a validated answer or confirmed correction may modify an application field.
INV-02	A doubt, command, failed recognition, or ambiguous reference may not be stored as an answer.
INV-03	A correction may modify only its explicit target field.
INV-04	An old-version result may not modify newer state or enqueue speech.
INV-05	Every product claim must have an approved fact or deterministic calculation.
INV-06	Spoken and written financial values must originate from the same structured projection.
INV-07	Stop and pause override reasoning, generation, queueing, and playback.
INV-08	Dependency failure must preserve confirmed application data and a valid checkpoint.
INV-09	The prototype cannot submit to a lender, approve, reject, sign, mandate, pay, or disburse a loan; the user-only completion marker has no external effect.
INV-10	The assistant cannot apply pressure, create urgency, hide costs, or treat decline negatively.
INV-11	The prototype cannot request or store real sensitive personal, banking, PAN, or Aadhaar data.
INV-12	Unheard generated content may not be represented as delivered conversation history.

6.2 Progressive fallback choices

The system selects the least disruptive route that restores correct behaviour:

1. discard obsolete work silently;
2. use a deterministic local alternative;
3. ask one focused clarification;
4. perform one bounded idempotent retry for a transient dependency;
5. degrade to shorter speech, written output, or supported text/keypad input;
6. abstain when evidence or safe interpretation is unavailable; or
7. pause or stop while preserving the last committed draft.

The assistant never retries when new user information is required and never continues speaking merely to conceal a dependency failure.

6.3 Failure decisions

Failure	Required safe outcome
Uncertain STT or interpretation	Ask for repetition/clarification; create no patch
Invalid field normalization	Explain the expected format; retain the same pending field
Cache unavailable	Bypass it; use direct Qdrant retrieval and the bundled approved local fact set
Missing or invalid product evidence	Abstain and offer the approved fact sheet
LLM timeout or invalid schema	One bounded repair/retry when safe, otherwise fixed response or abstention
Calculator error or stale projection	Withhold the affected value and preserve the previous confirmed draft
Rime or playback failure	Stop incomplete audio, mark it unheard, and offer safe repeat or written output
Storage conflict	Reject the patch, reload the current revision, and report no false success
Analytics failure	Buffer essential events; the run cannot pass until the trace is complete
Disconnect	Cancel active work and preserve the last committed draft/checkpoint

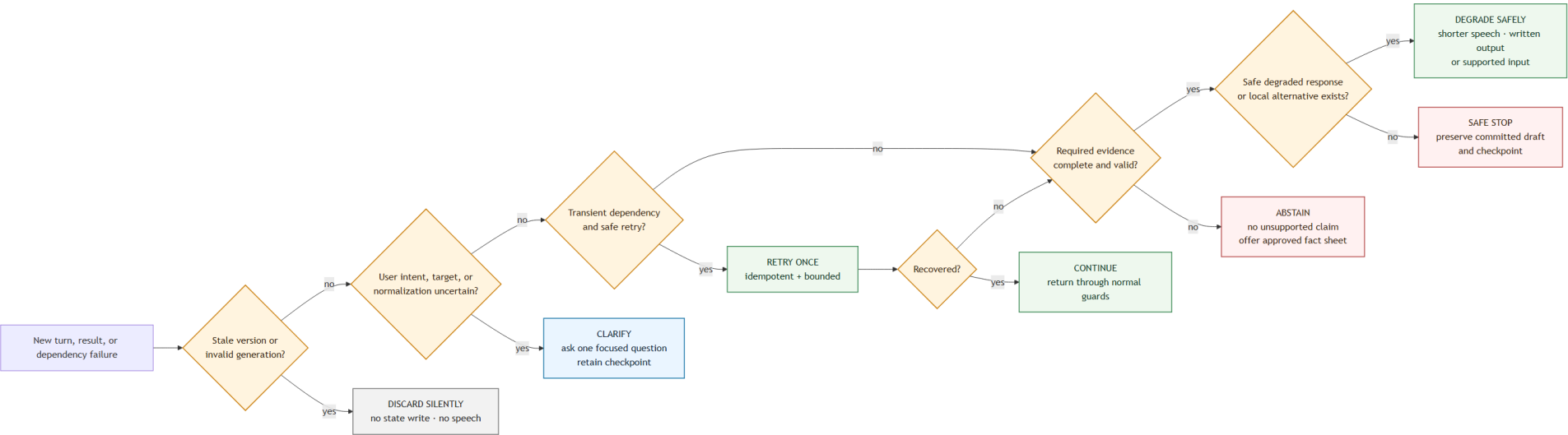


Figure 4: Simplified fallback decision tree. The runtime selects the least disruptive safe outcome: discard stale work, clarify uncertainty, retry a safe transient failure once, abstain when evidence is missing, degrade when possible, and otherwise preserve state and stop.

7 Research Evidence and Design Choices

7.1 How evidence is used

The studies benchmark uncertain AI behaviour and directly test deterministic contracts. They use synthetic, versioned fixtures and justify the prototype design; they do not establish production reliability, regulatory compliance, or population-level performance.

7.2 Literature basis for evaluated alternatives

The alternatives in the next table were selected from established research directions rather than being arbitrary implementation variants. The papers motivate the candidate method families; the experiment results below determine which candidate is retained for this prototype. The citations therefore provide the research basis for an alternative, while the measured result remains our own evidence.

TR	Research source(s)	How the literature informed the alternatives
TR-1	Rubin, Herzig, and Berant (2022), “Learning To Retrieve Prompts for In-Context Learning”	Prompt-example selection affects in-context performance. This motivated comparing zero-shot, fixed demonstrations, and retrieved demonstrations rather than treating prompting as one undifferentiated option.
TR-2	Wu et al. (2019), “Transferable Multi-Domain State Generator for Task-Oriented Dialogue Systems”	Dialogue-state tracking represents user turns as structured domain–slot–value state. This motivated comparing an unconstrained mutable pointer with an explicit typed, versioned state reducer.
TR-3	Lewis et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”; Niu et al. (2024), RAGTruth; Kamath, Jia, and Liang (2020), “Selective Question Answering under Domain Shift”	Retrieval can supply external evidence, but generated answers can still be unsupported and confidence can be overconfident under shift. This motivated closed-book, retrieval variants, evidence/claim checks, and abstention.
TR-4	Sproat and Jaitly (2017), “An RNN Model of Text Normalization”	Text normalization maps written forms to spoken forms, with numbers, dates, currencies, and units requiring unusually strict accuracy. This motivated comparing ordinary prose with a deterministic listener-oriented renderer and a fact-preservation gate.
TR-5	Selfridge et al. (2013), “Continuously Predicting and Processing Barge-in During a Live Spoken Dialogue Task”	Barge-in handling requires deciding whether to pause, continue, or resume while the user speaks. This motivated a cancel-first alternative that fences delayed work instead of stopping only local audio.
TR-6	Perez et al. (2022), “Red Teaming Language Models with Language Models”	Automated adversarial test generation can expose harmful behaviour that ordinary examples miss. This motivated comparing prompt-only policy compliance with deterministic release gates, fixed fallbacks, and an action allowlist.

7.3 Experiment summary

TR	Evaluated alternatives	Dataset or execution	Strongest observed result
TR-1	Zero-shot, fixed eight-shot, and retrieved five-shot interpretation [R1] with Groq and Gemini	120 labelled utterances: 37 demonstrations, 24 development, 59 held-out	Groq retrieved five-shot plus guard: 0/46 observed unsafe writes and 1.41 s p95; six of 13 safe writes were held.
TR-2	Mutable current-field pointer versus guarded versioned reducer [R2]	30 scripted conversations; 71 transitions per approach	Guarded reducer: 100% scenario/state metrics and zero unintended mutations.
TR-3	Closed-book, unscoped/scoped BM25, guarded alias routing, and optional second verification [R3]	32 facts, 50 English/Hinglish QA cases, 16 development generation cases	A3 recall@3 82.1%, zero stale/wrong-product intrusion; Groq A3 p95 3.59 s.
TR-4	Ordinary financial prose versus listener-oriented rendering before identical Rime synthesis [R4]	25 paired utterances, 45 facts, 50 Rime outputs and ASR round trips	Optimized: 45/45 facts preserved, 97.8% ASR fact recall versus 15.6% baseline; won 23/25 pairs.
TR-5	Cancel-first generation fencing under delayed speech and tool results [R5]	12 integration scenarios, four stop trials, real Rime PCM and virtual sink	All scenarios passed; 17.5 ms application stop p95; no obsolete re-entry.
TR-6	Policy-prompted model alone versus deterministic gates, fallback, and action allowlist [R6]	30 adversarial responses and 16 dispatcher tests	Raw model passed 11/30; guarded final output passed 30/30; all forbidden actions blocked.

7.4 Implementation reconciliation after live integration

The frozen TR-1 experiment above remains the historical model comparison: Groq retrieved five-shot produced the strongest offline result in that study. During integrated voice runs, however, repeated Groq interpreter failures made runtime continuity the more important engineering constraint. The implemented interpreter therefore uses deterministic fast paths for controls, confirmations, hedged monetary values, calculation cues, and plain pending-field answers, then calls Gemini for the remaining structured interpretation cases. Gemini still returns only an untrusted `TurnProposal`; explicitness checks, field normalizers, the guarded reducer, and provisional confirmation retain write authority. This provider switch is an implementation decision supported by live traces, not a retroactive change to the frozen offline benchmark.

Groq remains available only as an optional grounded-wording adapter. It is disabled by default because approved atomic product facts are already written for speech and avoiding a serial wording call reduces latency. The implemented Rime profile is Coda with the Nadi voice for the selectable English and Hindi output configurations, `speedAlpha=0.92`, 24 kHz audio, and websocket streaming.

7.5 Decisions carried into the MVP

TR	Selected method	Why this choice
TR-1	Deterministic fast paths followed by Gemini retrieved five-shot structured interpretation, explicitness and hedge gates, field-specific normalizers, and provisional confirmation	Preserves low latency for common controls and answers while using Gemini for genuinely ambiguous off-path turns; every model result remains a proposal.
TR-2	One typed, versioned guarded reducer with compare-and-set application commit	Outperformed a mutable pointer on correction, preservation, mixed turns, and stale results.

TR	Selected method	Why this choice
TR-3	Scoped and alias-expanded retrieval, deterministic calculation routing, claim–evidence mapping, support/completeness gates, and abstention; optional Groq wording is off by default	Prevents stale/wrong-product evidence while avoiding an unnecessary serial LLM call in the ordinary grounded path.
TR-4	Typed value–label pairs, deterministic listener renderer, fact-preservation gate, and Rime <code>coda/nadi</code> at 24 kHz	Dense correct prose was not reliably recoverable from audio; structured rendering improved measured intelligibility and the selected voice is consistent across both UI language profiles.
TR-5	Cancel first, increment generation, stop local sink, clear queue, then navigate or generate replacement output	Prevents both buffered speech and delayed reasoning from re-entering.
TR-6	Typed response plan, deterministic release gates, fixed fallbacks, and physically restricted action allowlist	Prompt-only policy compliance failed most adversarial cases.

7.6 Important rejected alternatives

- Direct LLM writes to application state.
- Prompt history or a single current-field pointer as workflow state.
- Unscoped retrieval or similarity score alone as an answerability rule.
- An always-on second verifier without demonstrated quality gain inside the latency budget.
- Sending dense financial prose directly to TTS.
- Cancelling only TTS or only the local playback queue.
- Relying on policy prompting as the sole safety boundary.

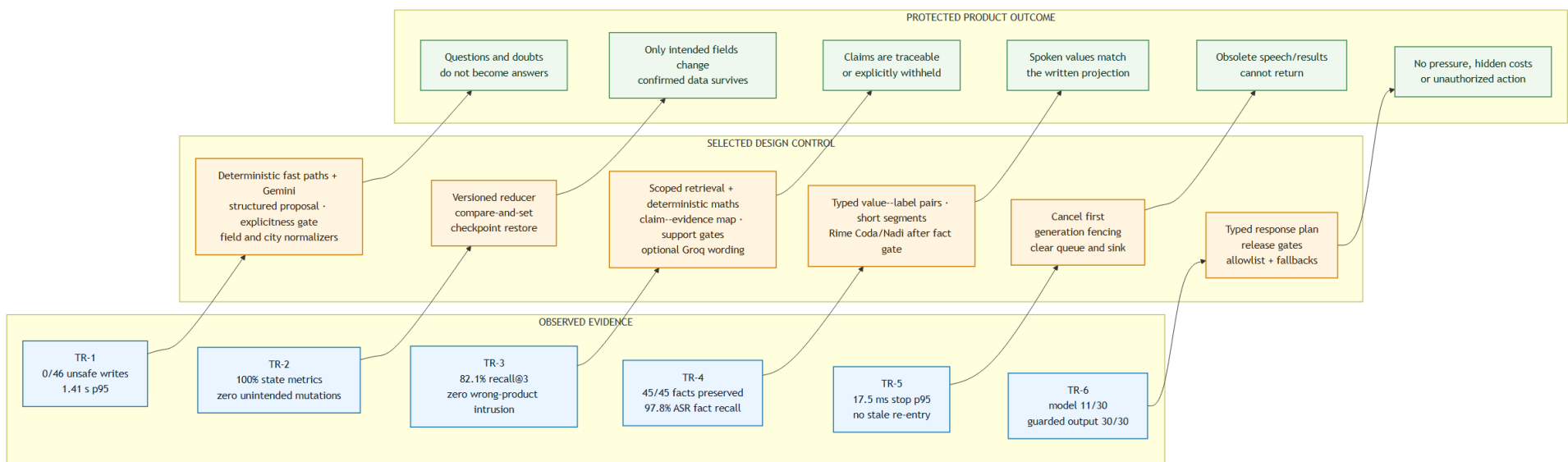


Figure 5: Research-to-design map. Each TR experiment supplies an observed result, a selected MVP control, and a protected product outcome; detailed experiment procedures remain outside the main design narrative.

8 Evaluation and Acceptance

8.1 Evaluation principle

The evaluation should show not only that the prototype completes a happy path, but that its claims are falsifiable. It therefore combines automated state and safety tests, provider integration tests, manual listening checks, and one observable end-to-end journey.

Hard safety and correctness gates are not averaged. One unintended mutation, stale re-entry, unsupported released claim, financial mismatch, cross-session leak, or prohibited action produces a failed release verdict. Missing evidence produces an inconclusive verdict rather than an assumed pass.

8.2 What is tested and how

Capability	Test evidence	Acceptance	Method
TR-1 interpretation	Frozen labelled turn set and integrated challenge turns	Report route/act errors separately; zero unsafe writes after guard	Automated plus targeted review
TR-2 state consistency	Normal, doubt, mixed, correction, navigation, failure, and stale-result scenarios	Exact resumption and final state; zero unintended mutation	Automated deterministic and integration tests
TR-3 grounding	Frozen QA set, Qdrant parity, fact-ID audit, and unanswerable cases	Zero wrong-product/stale intrusion and unsupported released claims	Automated retrieval/QA plus manual claim audit
TR-4 speech	Paired financial utterances, value-label comparison, Rime audio, and listener sheets	100% critical value-label agreement; acceptable intelligibility and pacing	Automated preservation/ASR plus two-listener review
TR-5 controls	Stop, pause, repeat, go-back, delayed-result, and queued-audio scenarios	Zero obsolete state/audio re-entry; measured user-facing stop latency	Automated integration plus live manual interruption
TR-6 neutrality	Adversarial response cases and dispatcher action attempts	All final release gates pass; every forbidden action blocked	Automated red-team gates plus team review
System trace	Event export and dashboard reconstruction for every final scenario	Required IDs/events complete; replay reproduces final state and verdict	Automated replay plus visual inspection
Session isolation	Two concurrent synthetic sessions with overlapping turn numbers	Zero cross-session read, write, cache, retrieval, or dashboard join	Automated integration test

8.3 Performance and reliability targets

Measure	Target	Interpretation
Normal end-of-turn to first audio	$p95 \leq 3 \text{ s}$	Provisional interaction target
Grounded end-of-turn to first audio	$p95 \leq 5 \text{ s}$	Provisional target, reported by cache hit and miss
User-facing stop latency	$p95 \leq 500 \text{ ms}$	Browser click-to-local-audio-pause timing; worker cancellation and stale-result fencing are evidenced separately
State integrity	Zero unintended/silent mutations	Hard gate
Stale protection	Zero obsolete state or playback re-entry	Hard gate
Grounding	Zero unsupported released product claims	Hard gate
Financial consistency	100% agreement across typed, written, and spoken value-label pairs	Hard gate
Trace completeness	100% required final-run evidence	Hard gate

8.4 Observed implementation snapshot

The following values were read from the append-only local SQLite trace after the integrated runtime evaluation on 7 September 2026. The evaluated path used Gemini, Deepgram, LiveKit, and Rime. The same event schema is now persisted in Railway PostgreSQL for deployed sessions, but the historical values below have not been relabelled as production measurements. They remain engineering results from synthetic development sessions, not an independent user study or production service-level claim.

Selected live acceptance trace. Session `session_b6f1930d...aa70e1` is retained because it contains both grounded questions and explicit browser stop measurements used in the previously reported acceptance table.

Measure	Observed	Samples	Interpretation
Normal/control response release p95	104.4 ms	6	Meets the provisional 3 s target for the observed fast-path turns
Grounded/product-question release p95	3,301.5 ms	2	Meets the provisional 5 s target
User-facing stop p95	0 ms	2	Recorded successfully; rounded browser timer result, not proof of zero acoustic latency
Turn trace completeness	8 accepted / 8 interpreted / 8 released	8 turns	Complete for the captured turns
Grounding and hard-gate outcome	2 supported decisions; 0 hard failures	2 grounded turns	Observed session verdict: PASS

All-session development aggregate. The dashboard contained 69 sessions, of which 50 had evaluable turns; 704 accepted turns and 654 response-latency samples were recorded. Interpreter completion was 85.7%, response latency was 1,648.3 ms p50 and 4,779.2 ms p95, and 89.0% of response samples met their route-specific 3 s or 5 s target. Three browser stop samples produced a rounded 0 ms p95 and 100% target attainment. The aggregate pass rate was 92.0%, with 52 failed-closed interpreter outcomes and zero recorded hard-failure events. These totals include debugging and partial sessions and therefore describe implementation telemetry rather than a clean benchmark cohort.

The current automated code suite also passes 95 backend tests, with five live-provider tests skipped by default, and six frontend tests. Automated success does not replace the bounded manual checks for intelligibility, audible stop behaviour, and the final demonstration path.

8.5 Canonical scenario set

The final suite covers fifteen observable situations:

1. normal field answer;
2. contextual field doubt;
3. product-cost doubt;
4. correction of the current field;
5. correction of an earlier field;
6. answer combined with a doubt;
7. ambiguous response;
8. low-confidence speech recognition;
9. repeat;
10. go back;
11. pause and resume;
12. stop during speech;
13. unsupported product question;
14. retrieval or model timeout; and
15. stale result after interruption.

All scenarios are automated at the transcript/state level. A smaller critical subset is repeated with live providers and real audio to verify latency, pronunciation, playback cancellation, and perceived continuity.

8.6 End-to-end demonstration

The live journey is intentionally compact:

1. start a clean synthetic draft and answer fields normally;
2. ask a contextual field doubt and show that the draft is unchanged before exact resumption;
3. correct an earlier answer and show that only the intended field changes;
4. ask a supported cost question and hear a short explanation with visible fact IDs;
5. interrupt active speech and show that stale audio/result cannot re-enter;
6. complete and review the draft and faithful financial summary;
7. manually mark the demo complete or decline, stop the voice worker, and show the explicit no-lender-submission outcome; and
8. open the dashboard to reconstruct the decision, evidence, versions, latency, and verdict.

8.7 Verdict

Verdict	Meaning
PASS	Every hard gate passes, required evidence is complete, the end-to-end journey succeeds, and provisional latency targets are met.
CONDITIONAL	Every hard gate passes and the safe path works, but a provisional latency or bounded listening target misses and is disclosed.
FAIL	At least one safety, state, grounding, financial, privacy, action-boundary, or isolation hard gate fails.
INCONCLUSIVE	Required scenarios, events, audio, artifacts, or evaluator outputs are missing or corrupted.

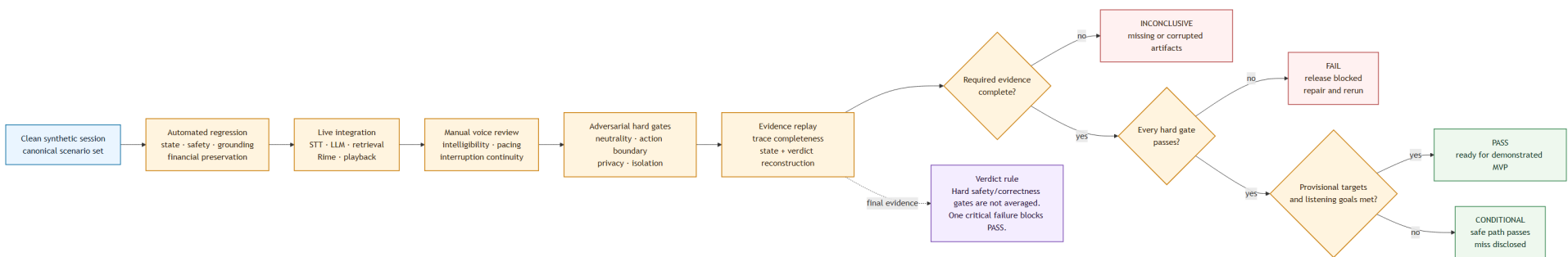


Figure 6: Simplified acceptance flow. Automated, live, manual, adversarial, and replay evidence feed hard release gates; missing evidence produces an inconclusive verdict, while a safe-path target miss is disclosed as conditional.

9 Observability and Demonstration Evidence

9.1 Why observability is part of the product proof

The dashboard is not a generic analytics feature. It demonstrates that the system preserved the pending field, rejected stale work, grounded a claim, committed only an authorized patch, and recorded what the borrower actually heard.

9.2 Minimum event envelope

Each material event carries session, trace, turn, timestamp, component, event type, application revision, state version, generation ID, outcome, latency, and a sanitized payload. Critical events add target field, patch decision, fact/calculation IDs, fallback reason, control outcome, and speech-delivery status where relevant.

9.3 Dashboard panels

Panel	Question it answers
Journey overview	Did the borrower complete, stop, or cancel, and what was the final draft status? The current session is selected by default and prior sessions remain selectable.
Application state	Which values were committed, corrected, or left unanswered?
Conversation timeline	Which turns were answers, doubts, corrections, controls, or fallbacks?
Grounding evidence	Which approved facts or calculations supported each released claim?
Control and stale-work evidence	How quickly did output stop, and which obsolete results were rejected? The selected-session stop row shows the observed p95 and sample count without repeating the target.
Voice delivery	Which response segments were generated, played, interrupted, or fully heard?
Safety and acceptance	Which hard gates passed or failed, what is the final verdict, and how do aggregate KPIs compare across all recorded sessions?

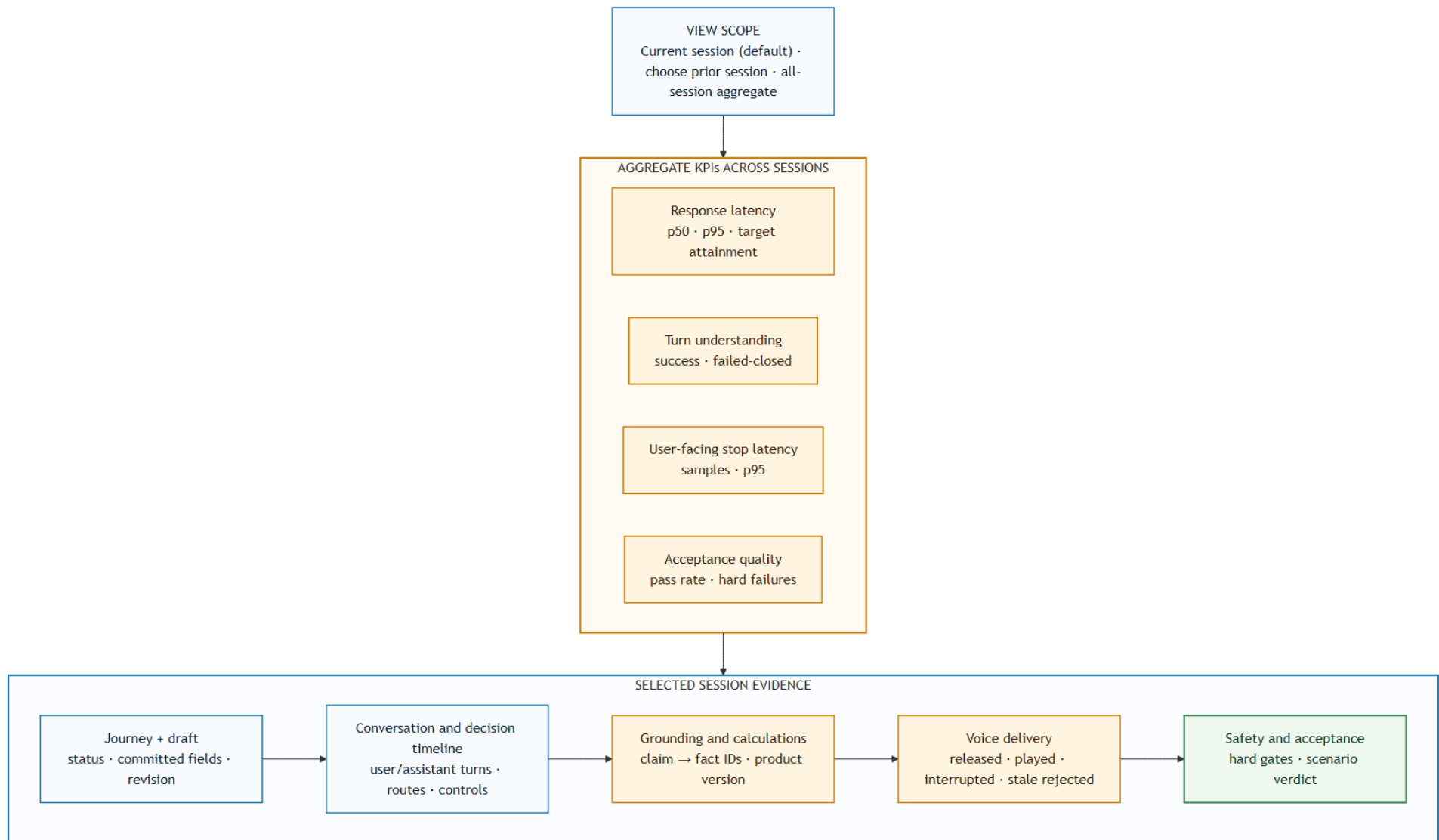


Figure 7: Dashboard wireframe. The default current-session view can switch to a prior session, while aggregate KPIs summarize performance across sessions. The selected-session panels expose draft state, the conversation and decision timeline, grounding, voice delivery, and the final safety verdict.

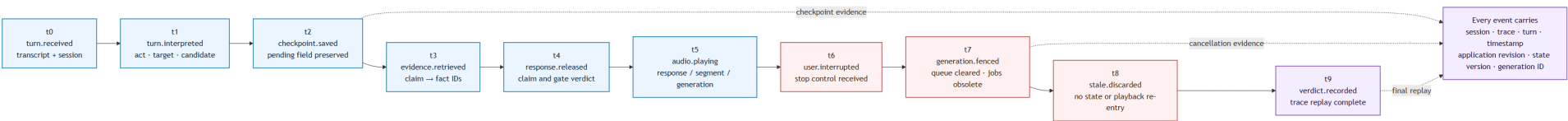


Figure 8: Event timeline. A representative turn is reconstructed from interpretation through grounding, release, playback, interruption, generation fencing, stale-result rejection, and final verdict recording.

10 Limitations and Future Work

10.1 Claims deliberately not made

The prototype does not establish suitability for real borrowers, regulated lending, accessibility compliance, production telephone networks, real customer data, credit decisions, or commercial conversion. The datasets are synthetic and bounded; manual listening involves only the project team; latency depends on the selected demonstration channel and providers.

10.2 Future work

Potential extensions include multilingual or code-switched interaction, multiple loan products, telephony transport, authenticated session recovery, human-agent handoff, broader accessibility studies, production privacy/compliance controls, and larger independent user evaluation. Real application submission, underwriting, and disbursal would require a separate product, safety, legal, and integration programme.

A Implementation Low-Level Design

A.1 Purpose and fixed boundaries

This appendix translates the approved product and technical requirements into implementation contracts. It is deliberately more specific than the high-level architecture, but it does not widen the product scope. The implementation remains a synthetic, anonymous, draft-only pre-application. Models may propose interpretations and wording; only deterministic components may commit application state, release financial claims, authorize actions, or decide whether work is still current.

A.2 Runtime processes and deployment units

Unit	Primary responsibility	State and trust boundary
Borrower web client (Vercel)	Microphone permission, LiveKit room connection, transcript display, playback, controls, and draft view	Holds presentation state only. It receives a short-lived room token and never receives provider secrets or direct database credentials.
FastAPI service (Railway)	Creates anonymous sessions, issues room-scoped LiveKit tokens, dispatches the named worker, returns draft/trace/verdict views, and exports the written review artifact	May initialize a session but does not bypass the guarded reducer for field changes. It exposes no submit, approve, reject, sign, pay, mandate, or disburse operation.
LiveKit agent worker (Railway)	Joins the LiveKit Cloud room, connects streaming STT and TTS, coordinates turns, handles interruption events, and publishes current audio	Holds temporary jobs and audio queues. It shares the production PostgreSQL repository with FastAPI and cannot commit a field except through the guarded reducer.
Guarded application core	Interprets finalized turns, validates patches, retrieves evidence, calculates values, releases safe responses, and emits trace events	Owns all business decisions. Domain modules are provider-independent and are shared by FastAPI and the agent worker.
Provider/data adapters	Deepgram, Gemini, optional Groq, Rime, Qdrant Cloud, optional Redis, and asynchronous SQLAlchemy integrations	Convert external responses into typed internal results. PostgreSQL is authoritative in production; SQLite remains the local-development fallback. No external provider receives mutation authority.

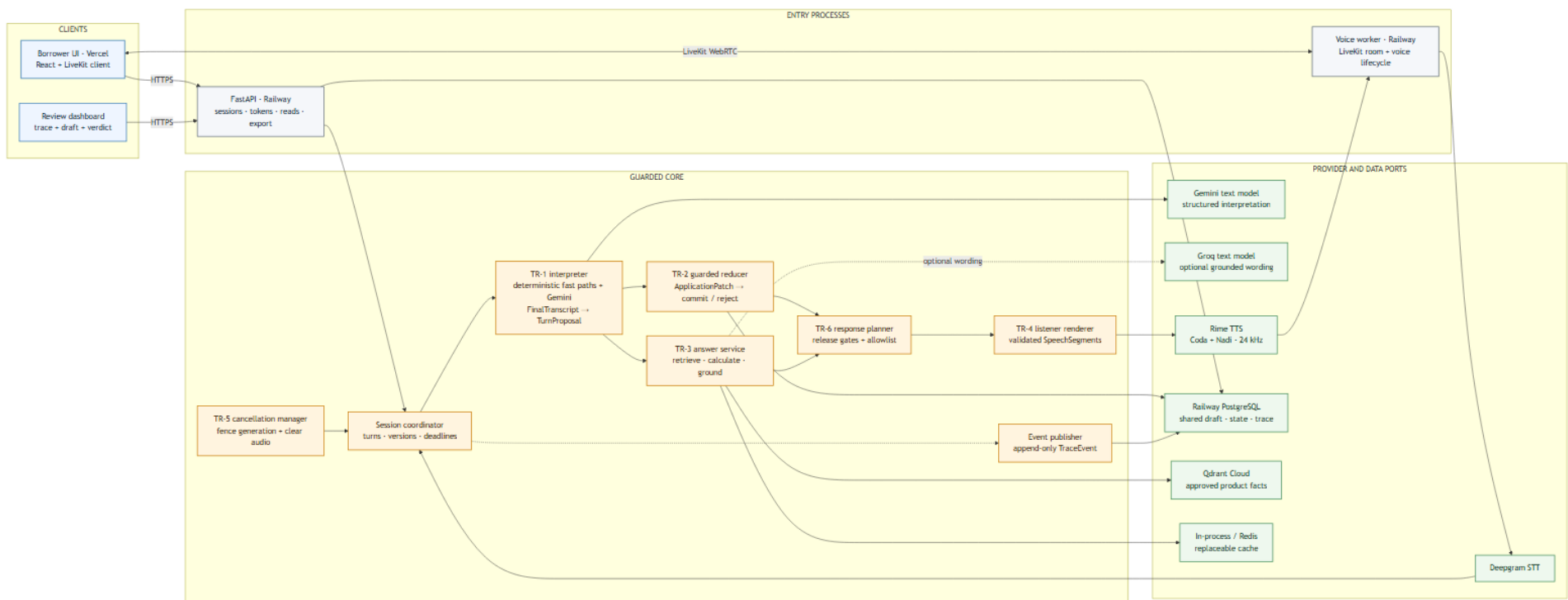


Figure 9: Implementation component map. The Vercel client enters through the Railway FastAPI service or Railway voice worker; the guarded core owns decisions; Railway PostgreSQL provides shared authoritative state and trace storage. Dotted output represents append-only trace publication.

A.3 Component contracts

Component	Consumes	Produces	Required invariant
Session API	New-session request or authenticated dashboard read	Anonymous IDs, room token, draft/trace/export views	Secrets remain server-side; reads are scoped by session and application ID.
Session coordinator	Final transcript, control event, provider result, current checkpoint	Route command, job registration, deadline, generation decision	Every asynchronous result is checked against session, turn, route, generation, and state revision.
Cancellation manager	Speech-start interruption, stop, pause, go-back, correction, or newer turn	Incremented generation, cancelled jobs, cleared audio, navigation command	Cancellation is established before replacement work; old work can neither speak nor write.
STT gateway	LiveKit audio frames tagged with session/participant	Interim transcript events and one finalized FinalTranscript	Interim text is display-only. Only a final, current transcript enters TR-1.
TR-1 interpreter	Final transcript plus bounded pending-field/checkpoint context	Typed TurnProposal : act, route, target, candidate, uncertainty	Deterministic controls and common answers bypass Gemini; model outputs remain proposals and doubts/controls cannot become answers.
TR-2 guarded reducer	ApplicationPatch , expected revision, idempotency key, confirmation status	Atomic commit result or typed rejection	At most one intended field changes; rejection has no partial effect; accepted commit increments revision exactly once.
TR-3 answer service	Product question, product/fact-set version, current synthetic values	GroundedAnswer with claims, fact IDs, calculations, and support status	Every released financial claim is supported by approved evidence or deterministic calculation; otherwise abstain.
Financial calculator	Typed amount, fee, rate, tenure, and calculator version	Labelled monetary projection with source revision	Pure deterministic function; output is invalidated when any dependent input changes.
TR-6 response planner	Commit/route result, grounded answer, checkpoint, allowed actions	Typed ResponsePlan or fixed safe fallback	Enforces grounding, cost completeness, numerical consistency, neutrality, draft status, and action allowlist.
TR-4 listener renderer	Approved response plan and labelled values	Ordered SpeechSegment records and written projection	Written and spoken values originate from one structured projection; the renderer cannot invent or alter a value.
Rime adapter	Current-generation speech segments and selected session language	Coda/Nadi tagged PCM/audio chunks at 24 kHz	Audio retains response, segment, and generation IDs and is published only while current.
Event publisher	Material decisions and provider timings	Append-only TraceEvent envelope	Events are immutable, session-scoped, and sufficient to reconstruct state and heard output.

A.4 Core message contracts

These messages form the boundaries between components. They should be implemented as Pydantic models and versioned independently of provider payloads.

Message	Minimum fields
FinalTranscript	<code>session_id</code> , <code>participant_id</code> , <code>turn_id</code> , <code>generation_id</code> , text, language, confidence, start/end timestamps, <code>is_final=true</code> .
TurnProposal	Proposal ID, act set, route, explicit target field, candidate value, reference resolution, uncertainty, rationale code, source transcript ID.
ApplicationPatch	Patch ID/idempotency key, application ID, expected application revision, source turn/generation, target field, normalized candidate, confirmation evidence.
CommitResult	Accepted/rejected, reason code, previous/new revision, changed field, previous/new value, checkpoint effect.
GroundedAnswer	Product and fact-set version, question route, claims, supporting fact IDs, calculation IDs, support/completeness result, abstention reason.
ResponsePlan	Purpose, message segments, labelled values, required disclosure set, allowed action, resume instruction, release-gate results, fallback code.
SpeechSegment	Response ID, segment ID/order, generation ID, text, value-label references, playback status, heard boundary, synthesis metadata.
TraceEvent	Event ID/type/time, session/application/trace IDs, turn/generation/application versions, component, decision/reason, latency, correlation and parent IDs.

A.5 Session creation and API surface

FastAPI creates the anonymous identity set and issues a short-lived LiveKit token. The route names below are implementation defaults; their semantics and authorization boundaries are fixed.

Method and route	Purpose	Draft mutation
POST <code>/api/sessions</code>	Create language-scoped <code>session_id</code> , <code>application_id</code> , participant identity, trace ID, empty revision-zero draft, and room name	Initialization only
GET <code>/api/sessions</code> and GET <code>/api/evidence/aggregate</code>	List selectable sessions and return cross-session dashboard KPIs	No
POST <code>/api/livekit/token</code>	Issue a short-lived token restricted to the assigned room and participant identity	No
GET <code>/api/sessions/{id}</code>	Return journey status, pending field, versions, and current checkpoint summary	No
GET <code>/api/sessions/{id}/draft</code>	Return the current synthetic review draft and written projection	No
GET <code>/api/sessions/{id}/events</code>	Return ordered trace events for dashboard reconstruction	No
GET <code>/api/sessions/{id}/acceptance</code>	Return release-gate and scenario verdict evidence	No
POST <code>/api/sessions/{id}/controls</code>	Carry a typed stop, pause, resume, repeat, go-back, summary, or cancel command from the UI into the same orchestrator path as voice control	Navigation or cancellation only

Method and route	Purpose	Draft mutation
POST /api/sessions/{id}/delivery	Record delivered or interrupted speech-segment state and heard-character boundary	No application mutation
POST/api/sessions/{id}/metrics/stop-latency	Append the browser click-to-local-audio-pause measurement to the trace	No
GET /api/sessions/{id}/export	Generate the written review artifact and approved synthetic fact sheet	No

There is deliberately no API route for loan submission, approval, rejection, signing, mandate creation, payment, or disbursal. The safest implementation of the draft-only boundary is for those capabilities not to exist.

The browser's `Submit` application control is intentionally outside this API surface. Once all eight synthetic fields are present, it publishes a room-local `submit` control that stops the voice worker and changes only the browser's demonstration status. It does not create a lender request, mutate the draft, or call an external system. The help-ticket control is likewise a presentation-only demonstration acknowledgement rather than a real human-service integration.

A.6 One finalized-turn execution

1. The browser publishes microphone audio to the assigned LiveKit room.
2. The agent streams tagged frames to Deepgram. Interim transcripts are observable but cannot trigger business logic.
3. A finalized transcript is accepted only if its session, turn, and generation are current.
4. TR-1 attempts deterministic confirmation, control, hedge, calculation, and plain-answer paths; remaining turns are interpreted by Gemini into a typed proposal.
5. An answer/correction becomes an application patch and passes through TR-2; a doubt routes to TR-3 without changing the draft; a control routes to TR-5; uncertainty produces clarification.
6. TR-6 builds a response plan from the route result and enforces deterministic release gates.
7. TR-4 creates short labelled segments from the approved plan and verifies fact preservation.
8. Rime returns tagged audio chunks. The agent publishes them only while their generation remains current.
9. Each decision and latency emits a trace event. Heard status is updated from actual playback and interruption events, not from generated text alone.

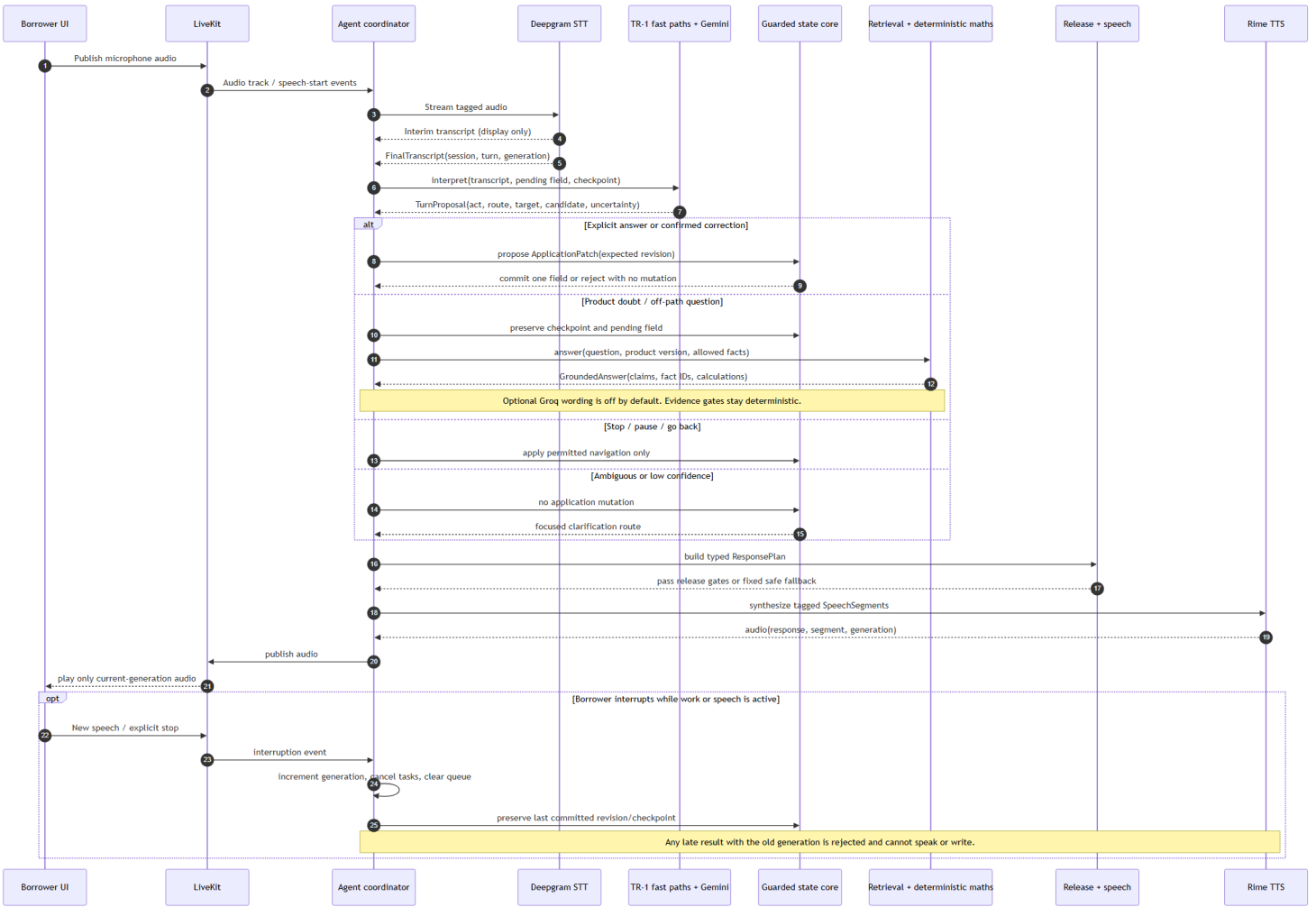


Figure 10: Low-level turn sequence. The same finalized transcript can route to a guarded field commit, grounded doubt resolution, permitted control, or clarification. Interruption creates a new generation boundary before any replacement work.

A.7 Persistence, cache, and retrieval design

Store	Contents	Rules
Railway PostgreSQL sessions	Anonymous IDs plus the serialized conversation state: status, pending field, resume checkpoint, navigation stack, active generation, output queue, heard response, and timestamps	One row per voice session; both production processes use this checkpoint store. It contains no real customer identity or sensitive lending data.
Railway PostgreSQL application_drafts	Application ID, revision, typed synthetic field values, calculator projection reference	Authoritative. Updates occur only through the reducer transaction and are immediately visible to the API and worker.
Railway PostgreSQL idempotency_results	Idempotency key, application ID, and serialized commit result	Makes duplicate commit attempts return the recorded outcome instead of applying a second mutation.
Railway PostgreSQL trace_events	Append-only event envelope and decision payload	Used by replay, dashboard, cross-session aggregation, latency calculation, and acceptance verdict.
Local SQLite fallback	The same logical tables and serialized contracts for development and automated tests	Selected through DATABASE_URL ; it is not the production store and cannot be independently used by two deployed services.
Qdrant loan_product_facts	Embedded fact text plus fact ID, product ID, product version, status, type, and language	Every query filters product/version/ approved ; retrieved content never writes application state.
In-process cache and optional Redis	Reusable product-fact retrieval results keyed by normalized query	Replaceable and bounded by capacity/TTL. Cache failure falls through to Qdrant and then the bundled approved local fact set. No personalized speech cache.
Versioned fact-sheet artifact	Human-reviewable synthetic facts, definitions, disclosures, formulas, and source version	Authoritative publishing source. Indexing creates or replaces the matching Qdrant product version.

The reducer transaction checks application ID, expected revision, idempotency key, source turn and generation, explicit target, normalization, and confirmation. It writes one field value and increments the application revision once; the orchestrator then persists the updated serialized conversation checkpoint and appends trace evidence. Qdrant and caches never participate in this transaction.

A.8 Concurrency, retry, and idempotency rules

1. The coordinator creates one `turn_id` for each finalized user turn and one `generation_id` for each currently audible response branch.
2. Every LLM, retrieval, calculation, safety, and TTS job captures the versions present at launch.
3. Returning results are accepted only when all captured versions still match. A mismatch is a normal stale rejection, not a retryable error.
4. Application commits use compare-and-set on `application_revision`; duplicate idempotency keys return the recorded result.
5. Only transient, idempotent provider failures receive one bounded retry. Missing user information, failed grounding, validation failure, and revision conflict never retry automatically.
6. Stop, pause, go-back, correction, and a newer user turn increment the generation before cancellation and replacement output.
7. A provider timeout cannot erase a checkpoint, mutate a confirmed field, or be disguised by unrelated filler speech.

A.9 Configuration and secret ownership

Deployment setting	Use
LIVEKIT_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET, LIVEKIT_AGENT_NAME	Room transport, server-generated participant tokens, and dispatch to the named <code>saarthi</code> worker
DEEPGRAM_API_KEY	Direct streaming STT adapter
GEMINI_API_KEY, GEMINI_MODEL	Structured interpretation for turns not resolved by deterministic fast paths
GROQ_API_KEY	Optional grounded phrasing; disabled in the default low-latency path
RIME_API_KEY, model/voice/language settings	Coda/Nadi websocket synthesis at the configured speed and 24 kHz sample rate
QDRANT_URL, QDRANT_API_KEY, QDRANT_COLLECTION	Version-filtered product-fact retrieval
DATABASE_URL	Authoritative SQL state and trace connection; both Railway runtime services reference the same PostgreSQL service. Provider-style <code>postgresql://</code> URLs are normalized to the asynchronous <code>asyncpg</code> driver.
REDIS_URL	Optional shared cache backing; empty in the current bounded deployment, with the in-process cache still available.
ENVIRONMENT, CORS_ORIGINS	Production-mode reporting and the explicit browser-origin allowlist for the Vercel frontend.
VITE_API_BASE_URL	Public FastAPI base URL compiled into the Vercel frontend; it is not a secret.

Railway variables are service-scoped. The API and worker each receive the server-side values needed by their runtime, including the same LiveKit project and PostgreSQL reference; Deepgram and Rime are exercised by the worker. Only the public LiveKit URL, FastAPI base URL, and a short-lived participant token may reach the browser. Secrets remain in backend environments and the committed repository contains placeholders only.

A.10 Implemented package layout and implementation order

```
src/saarthi/
  api/          sessions, LiveKit tokens, reads, export
  agent/        worker, STT events, turn handling, audio publication
  domain/       typed models, reducer, checkpoints, policies
  services/     interpreter, grounding, calculator, planner, renderer
  providers/    Gemini, optional Groq, Deepgram, Rime, Qdrant, cache adapters
  storage/      SQLAlchemy repository: PostgreSQL production, SQLite local
  scripts/      fact-sheet compilation and Qdrant ingestion
  tests/        unit, scenario, provider integration, replay
  web/src/      borrower UI, room controls, transcript, draft, dashboard
  data/
    product/    versioned synthetic fact sheet
```

The implemented order followed typed contracts and the guarded reducer, fact-sheet retrieval/calculation, FastAPI session APIs, the LiveKit/Deepgram input path, TR-1/TR-3/TR-6 services, Rime output, cancellation fencing, the borrower interface, and dashboard/replay integration. Later refinements added Gemini interpretation, deterministic fast paths, false-interruption recovery, browser stop measurement, speaker-labelled transcripts, user-only demo completion, a demonstration help ticket, selectable dashboard sessions, aggregate KPIs, branded UI assets, and conversational city normalization. Each layer retains tests derived from the corresponding research-to-design record.

Core implementation rule: provider responses are untrusted proposals. The reducer owns application writes; the grounding and release gates own claims; the generation fence owns audibility; the action registry owns capability.